Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

E. Case study KBM . . . 2
E.1. Bối cảnh bài toán . . . 2
E.1.1. Sáu vấn đề cần giải quyết . . . 2
E.1.2. Nhóm người dùng . . . 3
E.1.3. Ràng buộc thực tế . . . 3
E.2. Bốn trụ cột nghiệp vụ . . . 3
E.2.1. SQL Lab — chấm SQL tự động . . . 4
E.2.2. Network Lab — chấm lập trình mạng . . . 4
E.2.3. Course Management — quản lý đào tạo . . . 4
E.2.4. DevOps Lab — thực hành DevOps/Kubernetes . . . 5
E.2.5. Thành phần bổ trợ . . . 5
E.3. Quá trình tiến hóa — từ Monolith đến Microservices . . . 5
E.3.1. Giai đoạn 1: Independent Silo Monoliths (2016–2022) . . . 5
E.3.2. Giai đoạn 2: SQL Judge — sáng tạo từ ràng buộc (2023) . . . 6
E.3.3. Giai đoạn 3: Thiết kế Microservices (2024) . . . 6
E.3.4. Giai đoạn 4: Chuyển đổi và mở rộng (2025) . . . 6
E.3.5. Giai đoạn 5: Polyglot — DevOps Lab (2025–2026) . . . 7
E.4. Kiến trúc hiện tại . . . 7
E.4.1. Bounded Context . . . 7
E.4.2. Công nghệ sử dụng . . . 8
E.4.3. Danh mục dịch vụ . . . 8
E.4.4. Tích hợp hệ thống bên ngoài . . . 9
E.4.5. Mô hình triển khai . . . 10
E.5. Trade-off và khoảng trống — bài học từ thực tế . . . 10
E.5.1. Nghiêm trọng (Critical) — 8 mục (tính đúng đắn / bảo mật) . . . 10
E.5.2. Quan trọng (Major) — 11 mục (khả năng mở rộng / bảo trì) . . . 10
E.5.3. Nhỏ (Minor) — 6 mục (cải thiện chất lượng) . . . 11
E.6. Sự cố thực tế và bài học . . . 11
E.6.1. Sự cố 1: Bài nộp trùng lặp trong kỳ thi . . . 11
E.6.2. Sự cố 2: Dữ liệu không nhất quán giữa các bảng . . . 12
E.6.3. Sự cố 3: Multi-instance Judge — định tuyến sai . . . 12
E.6.4. Sự cố 4: Gộp danh tính (Identity Merge) — khủng hoảng danh tính . . . 12
E.7. Ánh xạ giữa case study và các chương . . . 13

PHỤ LỤC E

Case study KBM

Phụ lục này trình bày đầy đủ về **KBM** — case study xuyên suốt cuốn sách. Từ bối cảnh bài toán ban đầu, qua quá trình tiến hóa kiến trúc kéo dài gần một thập kỷ, đến kiến trúc microservices tại thời điểm biên soạn cùng những trade-off và sự cố đã xảy ra trong thực tế.

Mục đích không chỉ minh họa lý thuyết, mà cho thấy kiến trúc phần mềm là *kết quả của một chuỗi quyết định trong ràng buộc thực tế* — không phải bản thiết kế lý tưởng trên giấy. Mỗi chương trong sách đều liên hệ với ít nhất một khía cạnh của KBM; phụ lục này tập hợp toàn bộ bức tranh để người đọc có thể đối chiếu và rút ra bài học riêng.

E.1. Bối cảnh bài toán

Tại một trường đại học kỹ thuật, các môn học về cơ sở dữ liệu, lập trình mạng, và lập trình thuật toán đòi hỏi sinh viên thực hành với khối lượng lớn bài tập — hàng trăm bài SQL mỗi tuần, hàng chục bài lập trình mạng Java, và các kỳ thi lập trình trực tuyến. Quy trình truyền thống (sinh viên nộp file, giảng viên chấm thủ công) nhanh chóng trở thành nút thắt khi quy mô tăng lên hàng nghìn sinh viên mỗi học kỳ.

E.1.1. Sáu vấn đề cần giải quyết

1. **Chấm thủ công không khả thi.** Giảng viên không thể thực thi và kiểm tra kết quả hàng trăm bài SQL trên nhiều hệ quản trị cơ sở dữ liệu (DBMS) khác nhau trong thời gian hợp lý. Mỗi bài tập cần chạy trên MySQL, PostgreSQL, SQL Server, hoặc Oracle — mỗi hệ có cú pháp và hành vi riêng.
2. **Phản hồi chậm.** Sinh viên thường phải chờ vài ngày mới nhận được kết quả, làm giảm hiệu quả học tập. Vòng lặp *viết — nộp — chờ — sửa* kéo dài thay vì phản hồi tức thì.
3. **Đa dạng CSDL.** Chương trình đào tạo yêu cầu thực hành trên nhiều DBMS. Tại thời điểm đó, nhóm không tìm được giải pháp tự vận hành được (nhất là mã nguồn mở) hỗ trợ đủ MySQL, PostgreSQL, SQL Server và Oracle trong cùng một nền tảng cho nhu cầu giảng dạy — đây là khoảng trống KBM phải tự lấp.

4. **Thiếu môi trường sandbox.** Sinh viên cần chạy mã SQL trực tiếp trên cơ sở dữ liệu thật để kiểm tra kết quả, nhưng không thể cho phép truy cập tự do vào hệ thống production — cần cơ chế cô lập (sandbox) ở mức container.
5. **Rủi ro gian lận thi cử.** Các kỳ thi trực tuyến trên phòng máy dễ bị sao chép kết quả giữa các máy nếu không có cơ chế giám sát và khóa môi trường (lockdown) tại máy khách.
6. **Quản lý phân tán.** Thông tin khóa học, bài tập, điểm số, và lịch thi nằm rải rác trên email, bảng tính Excel, và Google Drive — không có hệ thống tập trung, dẫn đến mất đồng bộ và thiếu truy xuất.

E.1.2. Nhóm người dùng

KBM phục vụ ba nhóm chính: **sinh viên** (hàng nghìn mỗi học kỳ — thực hành, thi, nộp bài tập, đồ án), **giảng viên** (vài chục — tạo đề, tổ chức thi, chấm điểm, theo dõi tiến độ), và **quản trị viên** (vài người — quản lý tài khoản, cấu hình hệ thống, giám sát vận hành).

E.1.3. Ràng buộc thực tế

Bối cảnh phát triển KBM chịu những ràng buộc đáng kể — chính các ràng buộc này định hình mọi quyết định kiến trúc:

- **Nhân sự hạn chế:** Nhóm phát triển chính chỉ 1–2 người, cộng thêm sinh viên hỗ trợ theo từng đợt đồ án.
- **Hạ tầng giới hạn:** vài máy chủ ảo (VPS), ngân sách phần cứng hạn chế — không có môi trường cloud hay cluster lớn.
- **Phát triển song song với giảng dạy:** Hệ thống được xây dựng *trong khi vẫn phục vụ giảng dạy thực tế*, không có giai đoạn "đóng cửa để xây lại".
- **Tích hợp hệ thống cũ:** Phải kết nối với hệ thống quản lý đào tạo (QLĐT) của trường — một hệ thống legacy không có tài liệu API chính thức.

Ý nghĩa kiến trúc

Những ràng buộc này giải thích tại sao nhiều quyết định kiến trúc trong KBM là *trade-off có chủ đích* thay vì thiết kế lý tưởng. Ví dụ: dùng shared database thay vì database-per-service (xem §E.5) không phải vì thiếu hiểu biết, mà vì nhóm nhỏ chưa cần tách máy chủ database; nhóm vẫn cần quy ước chủ sở hữu ghi, migration owner và ranh giới schema trong cùng PostgreSQL (Chương 7). Đây là thực tế mà sách kiến trúc ít khi nói tới.

E.2. Bốn trụ cột nghiệp vụ

KBM được tổ chức quanh bốn trụ cột nghiệp vụ chính, mỗi trụ cột tương ứng với một nhóm Bounded Context riêng biệt. Ngoài ra, hệ thống còn hai thành phần bổ trợ quan trọng.

Bảng E.1: *Bốn trụ cột nghiệp vụ KBM và yêu cầu kiến trúc đặc trưng*

Trụ cột	Chức năng chính	Yêu cầu kiến trúc đặc trưng
SQL Lab	Chấm SQL tự động trên 4 DBMS, 2 chế độ (luyện tập đồng bộ + thi bất đồng bộ), phản hồi AI	Sandbox cô lập mỗi DBMS, xử lý bất đồng bộ qua message broker khi thi
Network Lab	Chấm lập trình mạng Java: 6 nhóm giao thức (TCP → gRPC), ~20 cổng	Standalone Java runtime, plugin system, khởi động nhanh
Course Management	Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh, GitHub Classroom	CRUD + tích hợp bên ngoài (QLĐT, GitHub API)

Trụ cột	Chức năng chính	Yêu cầu kiến trúc đặc trưng
DevOps Lab	Thực hành K8s/Docker: container-in-container, web terminal, chấm tự động	Ngôn ngữ Go, k3s + Sysbox, wildcard DNS

E.2.1. SQL Lab — chấm SQL tự động

SQL Lab là trụ cột phức tạp nhất về mặt kiến trúc. Các online judge phổ biến được thiết kế cho ngôn ngữ lập trình truyền thống (compile → run → compare output), trong khi SQL không tuân theo mô hình này. KBM giải quyết bằng cách: *thực thi câu SQL trên sandbox CSDL thật, so sánh kết quả qua băm SHA-256 của result set*.

Hệ thống hỗ trợ hai chế độ vận hành:

- **Chế độ luyện tập (Practice Mode, đồng bộ):** Sinh viên gửi câu SQL → hệ thống thực thi trên sandbox tương ứng (MySQL, PostgreSQL, SQL Server, hoặc Oracle) → so sánh kết quả → trả phản hồi tức thì. Bài luyện tập nhỏ thường có kết quả trong vài giây; bài nặng hoặc đề chạy trên nhiều DBMS mất 5–30 giây (§4.1). Tích hợp phản hồi bằng AI (GPT-4o-mini) để giải thích lỗi logic.
- **Chế độ thi (Contest Mode, bất đồng bộ qua Kafka):** Trong kỳ thi, hàng trăm sinh viên gửi bài đồng thời. Hệ thống đẩy bài nộp vào Kafka topic → coordinator `kbm-judge-sql` tiêu thụ và phân bổ cho các worker `kbm-sandbox-*` chấm → kết quả đẩy lại qua WebSocket. Bước đưa vào hàng đợi (enqueue) rất nhanh so với thời gian chấm (mục tiêu p99 dưới 500 ms, §1.5), sinh viên không bị chặn chờ kết quả.

Kết quả chấm (verdict) bao gồm: AC (Accepted), WA (Wrong Answer), TLE (Time Limit Exceeded), RTE (Runtime Error), CE (Compilation Error) — cùng hệ thống phán định với các online judge truyền thống.

Cơ chế sandbox cô lập dữ liệu bằng tiền tố bảng (table prefix), kết hợp sắp xếp topo (topological sort) để thiết lập schema đúng thứ tự phụ thuộc.

E.2.2. Network Lab — chấm lập trình mạng

Network Lab phục vụ thực hành lập trình mạng Java, bao gồm 6 nhóm giao thức — từ socket cơ bản đến gọi thủ tục từ xa hiện đại:

1. **TCP** — 6 loại (echo, file transfer, multi-thread, v.v.)
2. **UDP** — 3 loại (datagram, multicast, broadcast)
3. **RMI** — 4 loại (registry lookup, callback, dynamic proxy, security policy)
4. **SOAP** — Web service cổ điển
5. **REST** — HTTP API
6. **gRPC** — Framework RPC hiệu năng cao

Điểm đặc biệt: sinh viên viết chương trình Java client trên IDE cá nhân, kết nối tới judge server qua giao thức tương ứng để nhận dữ liệu, xử lý và gửi đáp án. Judge là ứng dụng *standalone Java* (không dùng Spring Boot), dùng cơ chế *dynamic handler reloading* — một dạng plugin system cho phép tải lại handler của câu hỏi mà không cần khởi động lại judge service. Hệ thống phân bổ ~20 cổng mạng riêng biệt cho các nhóm giao thức.

E.2.3. Course Management — quản lý đào tạo

Trụ cột này bao quát các nghiệp vụ quản lý đào tạo truyền thống, nhưng với tích hợp sâu vào hệ sinh thái KBM:

- **Quản lý khóa học:** Tạo, cấu hình, phân quyền giảng viên.

- **Bài tập:** Hỗ trợ cả bài tập cá nhân và nhóm, định dạng rubric chấm điểm.
- **Thi trắc nghiệm (MCQ):** Ngân hàng câu hỏi, tạo đề ngẫu nhiên, chấm tự động.
- **Đồ án tốt nghiệp:** Quy trình 2 cấp duyệt → bảo vệ → chấm rubric trước hội đồng.
- **Điểm danh:** Theo buổi học (session-based), tích hợp lịch học.
- **Grade Scheme và CLO:** Cấu trúc điểm theo thành phần, ánh xạ chuẩn đầu ra (CLO — Course Learning Outcome).
- **GitHub Classroom:** Đồng bộ bài tập, liên kết danh sách lớp (roster), theo dõi tiến độ commit.

E.2.4. DevOps Lab — thực hành DevOps/Kubernetes

DevOps Lab — trụ cột mới nhất — được phát triển bằng **Go** thay vì Java, một quyết định có chủ đích:

- **Binary nhỏ gọn:** ~15 MB (Go binary) so với 200 MB+ (Spring Boot JAR + JRE).
- **Khởi động nhanh:** Mili-giây thay vì vài giây.
- **Goroutine:** Mô hình xử lý song song gọn nhẹ, giúp reverse proxy phục vụ hiệu quả nhiều kết nối đồng thời.
- **client-go SDK:** Tương tác native với Kubernetes API.

Kiến trúc sử dụng **mono-repo multi-binary**: một repository Go sinh ra 3 binary riêng biệt (`api`, `router`, `grader`). Hạ tầng dựa trên **k3s** (Kubernetes nhẹ) kết hợp **Sysbox** — runtime cho phép container-in-container an toàn mà không cần chế độ đặc quyền (privileged). Mỗi sinh viên nhận một *Lab Pod* cô lập với TTL (Time-To-Live) tự động thu hồi tài nguyên.

E.2.5. Thành phần bổ trợ

Ngoài bốn trụ cột chính, KBM có hai thành phần bổ trợ quan trọng:

Code Judge (`kbm-judge-code`) — Chấm thuật toán đa ngôn ngữ (C/C++/Java). Thể hiện rõ quyết định "xây hay mua" (build vs buy): thế hệ đầu tiên (Gen 1) được phát triển nội bộ nhưng **không có sandbox** — mã sinh viên chạy trực tiếp trên server, tạo rủi ro bảo mật nghiêm trọng. Thế hệ thứ hai (Gen 2) chuyển sang tích hợp **DOMjudge** — hệ thống chấm mã nguồn mở chuẩn ICPC, cung cấp sandbox đầy đủ. Quyết định này minh họa bài học xây hay mua trong Chương 3 và Chương 10.

Desktop Exam Client (`kbm-exam-client`) — Ứng dụng máy tính để bàn dành cho phòng máy thi, thực hiện *lockdown* môi trường: chặn ứng dụng ngoài, giới hạn IP cho phép (whitelist), và giám sát (proctoring) tại máy khách. Client sử dụng chung JWT token với `kbm-auth`, đảm bảo nhất quán xác thực.

E.3. Quá trình tiến hóa — từ Monolith đến Microservices

KBM không được thiết kế microservices ngay từ đầu. Hệ thống hiện tại là kết quả của gần một thập kỷ tiến hóa, qua 5 giai đoạn (era) — mỗi giai đoạn bắt đầu bằng một vấn đề thúc đẩy (trigger) và kết thúc bằng một bài học kiến trúc.

E.3.1. Giai đoạn 1: Independent Silo Monoliths (2016–2022)

Trigger: Nhu cầu ban đầu về chấm bài lập trình và quản lý đào tạo.

Giai đoạn đầu tiên không phải là một monolith duy nhất, mà là *ba hệ thống MVC rời rạc* — ba silo độc lập, mỗi silo là một monolith:

1. **Hệ thống quản lý đào tạo** — Monolith ASP.NET MVC, quản lý khóa học, bài tập, điểm số, tài khoản người dùng.
2. **Code Judge (Gen 1)** — Hệ thống chấm thuật toán C/C++ và Java, tự phát triển. Đặc điểm: *không có sandbox* — mã sinh viên chạy trực tiếp trên server.

3. **Network Judge** — Monolith MVC riêng biệt cho chấm lập trình mạng TCP/UDP/RMI.

Ba hệ thống hoạt động độc lập: không chia sẻ dữ liệu, không chia sẻ danh tính người dùng, không giao tiếp với nhau. Thomas Erl gọi đây là kiến trúc *silo-based* — mỗi ứng dụng là một "ốc đảo" với logic trùng lặp và chi phí bảo trì cao.

Bài học: Ba hệ thống rời rạc (independent silos) không phải distributed monolith — chúng không bị ràng buộc đồng loạt (lockstep) mà hoàn toàn cô lập. Tuy nhiên, sự cô lập này cũng không mang lại lợi ích của kiến trúc phân tán: logic trùng lặp, danh tính người dùng không liên thông, và chi phí bảo trì *nhân lên theo số hệ thống* thay vì được chia sẻ.

E.3.2. Giai đoạn 2: SQL Judge — sáng tạo từ ràng buộc (2023)

Trigger: Nhu cầu chấm SQL tự động — nhóm không tìm được sản phẩm tự vận hành được đáp ứng đủ.

Đây là cột mốc quan trọng nhất trong lịch sử phát triển KBM. Các online judge phổ biến được thiết kế cho ngôn ngữ lập trình (compile → run → compare), nhưng SQL không tuân theo mô hình này — không có stdin/stdout, kết quả là *result set* phụ thuộc vào dữ liệu mẫu.

Giải pháp: thực thi câu SQL trên sandbox CSDL thật (mỗi DBMS một container riêng), so sánh result set bằng băm SHA-256. Cơ chế cô lập sử dụng tiền tố bảng kết hợp sắp xếp topo cho các phụ thuộc của schema. Hệ thống hỗ trợ đồng thời 4 DBMS.

Bài học: Đổi mới thường sinh ra từ ràng buộc. Khi không có giải pháp có sẵn, việc thiết kế từ đầu (greenfield) buộc phải suy nghĩ sâu về bài toán.

E.3.3. Giai đoạn 3: Thiết kế Microservices (2024)

Trigger: Sự phức tạp gia tăng khi tích hợp SQL Judge vào hệ thống hiện có.

Kiến trúc microservices được áp dụng, nhưng không phải bằng một cuộc chuyển đổi Big Bang (viết lại toàn bộ). Thay vào đó, KBM phát triển theo kiểu *organic growth* (tăng trưởng hữu cơ): thiết kế microservices cho SQL Judge mới, rồi dần dần tách các thành phần cũ.

Dấu hiệu của sự phát triển hữu cơ: `spring.application.name = "app"` từng giống nhau giữa `kbm-core` và `kbm-academic` — bằng chứng rằng hai service này từng là một (Ch.4). Shared database `core_db` là *accepted transitional debt* (nợ kỹ thuật chuyển tiếp có chủ đích): đội ngũ nhỏ, cùng chu kỳ triển khai, ưu tiên tốc độ phát triển hơn tách biệt hoàn toàn.

Bài học: Microservices trong thực tế hiếm khi bắt đầu "sạch". Sự tiến hóa hữu cơ để lại dấu vết (architectural debt), và nhận diện được các dấu vết này là bước đầu tiên để cải thiện (xem chi tiết tại §E.5).

E.3.4. Giai đoạn 4: Chuyển đổi và mở rộng (2025)

Trigger: Nhu cầu tách biệt nghiệp vụ đang chồng chéo và mở rộng tính năng mới.

Giai đoạn này đánh dấu loạt thay đổi lớn:

- **Tách service:** `kbm-academic` được tách ra từ `kbm-core`, tuy vẫn dùng chung database (shared DB, giai đoạn chuyển tiếp).
- **Kafka:** Triển khai message broker cho chế độ thi, giải quyết bài toán giờ cao điểm — hàng trăm lượt nộp mỗi phút vào cuối giờ thi.
- **Network Judge mở rộng:** TCP/UDP/RMI → thêm SOAP → REST → gRPC, phản ánh thay đổi chương trình đào tạo.

- **Xây → Mua:** Code Judge Gen 1 (tự phát triển, không sandbox) → Gen 2 (DOMjudge, mã nguồn mở chuẩn ICPC).
- **Desktop Exam Client:** Ứng dụng lockdown cho phòng máy thi.
- **OAuth2 mở rộng:** Thêm Microsoft OAuth2, xác thực email, Cloudflare Turnstile.
- **Observability:** metric Prometheus, định danh tương quan / truy vết (chưa xuyên suốt — xem trade-off tại §E.5).

Bài học: Chuyển đổi không phải sự kiện một lần mà là quá trình liên tục. Mỗi bước chuyển đổi phải mang lại giá trị ngay lập tức, không chờ đến khi "chuyển đổi xong".

E.3.5. Giai đoạn 5: Polyglot — DevOps Lab (2025–2026)

Trigger: Tài nguyên VPS hạn chế, Java quá nặng cho workload proxy/grading.

Quyết định sử dụng Go thay vì Java cho DevOps Lab là minh chứng cho nguyên tắc *right tool for the right job*. Với hạ tầng giới hạn, binary Go 15 MB khởi động trong mili-giây là lựa chọn thực dụng hơn Spring Boot JAR 200 MB+.

Kiến trúc mono-repo multi-binary (1 repo → 3 binary: `api`, `router`, `grader`) khác biệt hoàn toàn so với multi-repo microservices của LMS Java. Sự khác biệt này minh họa rằng "microservices" không đồng nghĩa với "mỗi service một repo" — cấu trúc repo phụ thuộc vào ngữ cảnh đội ngũ và ngôn ngữ.

Bài học: Polyglot architecture (đa ngôn ngữ) mang lại lợi thế kỹ thuật, nhưng đòi hỏi đội ngũ phải thành thạo nhiều ngôn ngữ — một chi phí phải được đánh giá rõ ràng (xem trade-off tại §E.5).

Dòng thời gian tổng quan

2016: Independent Silo Monoliths (3 MVC) → **2023:** SQL Judge (cột mốc sáng tạo) → **2024:** Microservices (organic growth) → **2025:** Chuyển đổi + Kafka + DOMjudge → **2025–2026:** Polyglot Go DevOps Lab. Tổng cộng khoảng 10 năm phát triển, microservices chính thức từ 2024.

E.4. Kiến trúc hiện tại

E.4.1. Bounded Context

Kiến trúc KBM tổ chức theo 9 Bounded Context (BC), gồm 7 core và 2 supporting:

1. **Identity & Access** (`kbm-auth`) — Quản lý danh tính, RBAC, JWT, OAuth2 (Google/Microsoft), SSO.
2. **SQL Practice** (`kbm-core` + `kbm-judge-sql` + các worker `kbm-sandbox-*`) — Chấm SQL trên 4 DBMS.
3. **Network Practice** (`kbm-judge-network`) — Chấm lập trình mạng 6 nhóm giao thức.
4. **Code Practice** (`kbm-judge-code`) — Chấm thuật toán, tích hợp DOMjudge.
5. **Academic Management** (`kbm-academic`) — Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh.
6. **Examination & Proctoring** (logic trong `kbm-core`, mục tiêu tách thành `kbm-exam`) — Quản lý kỳ thi, chống gian lận, ứng dụng khóa môi trường thi (`kbm-exam-client`).
7. **DevOps Practice** (`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`) — Nền tảng thực hành DevOps bằng Go.
8. **Communication** (`kbm-notification`) — Supporting BC: đẩy thông báo thời gian thực qua WebSocket, tiêu thụ sự kiện từ Kafka.
9. **Platform / Infrastructure** (`kbm-gateway`, `kbm-registry`, `kbm-shared`) — Supporting BC: Gateway, service discovery, thư viện dùng chung (shared library).

⚠ God Service

`kbm-core` hiện chứa cả SQL Practice (BC #2) lẫn Examination (BC #6) — đây là ví dụ điển hình của anti-pattern *God Service* (xem §E.5). Lưu ý rằng `kbm-core` phát triển hữu cơ từ giai đoạn 3, khi ranh giới BC chưa được xác định rõ. Mục tiêu tương lai là tách phần thi cử thành `kbm-exam` riêng biệt.

E.4.2. Công nghệ sử dụng

KBM là hệ thống **polyglot** — hai nền tảng công nghệ khác nhau cho hai nhóm workload:

Bảng E.2: *So sánh tech stack giữa LMS chính (Java) và DevOps Lab (Go)*

Khía cạnh	LMS chính (Java)	DevOps Lab (Go)
Ngôn ngữ	Java 17 + Spring Boot 3.x	Go 1.x + Chi v5
Kiến trúc repo	Multi-repo microservices (hơn 10 repo)	Mono-repo multi-binary (1 repo → 3 binary)
Database	PostgreSQL + MySQL + SQL Server (JPA)	PostgreSQL (sqlx raw SQL)
Auth	Spring Security + JWT	Tại thời điểm khảo sát: shared JWT từ `kbm-auth` (HS256 shared secret)
Runtime	Docker Compose	k3s + Sysbox
Công cụ migration	JPA auto-DDL	golang-migrate
Frontend	2 ứng dụng (React + TS + Vite)	1 ứng dụng (kế thừa stack LMS)

Bảo mật của case study tại thời điểm khảo sát (đầu năm 2026): KBM dùng Gateway edge validation, `X-User-*` dẫn xuất và HS256 shared secret; một số service tự phân tích JWT. Đây là rủi ro được chấp nhận (accepted risk) kèm trôi lệch triển khai (implementation drift), không phải kiến trúc mục tiêu. Mục tiêu Tier 1 là RS256/JWKS, signed proof đúng audience tại mỗi resource service và phân quyền cục bộ; luồng nhạy cảm bổ sung workload identity cùng subject/actor/delegation theo Tier 2.

E.4.3. Danh mục dịch vụ

Bảng E.3 liệt kê các dịch vụ của KBM theo bounded context.

Bảng E.3: *Danh sách dịch vụ KBM theo Bounded Context*

Tên service kỹ thuật	Bounded Context	Mô tả vai trò
`kbm-auth`	Identity & Access	Quản lý danh tính người dùng, đăng nhập, phân quyền (RBAC), cấp phát và xác minh JWT, OAuth2 (Google/Microsoft), SSO.
`kbm-core`	SQL Practice + Examination	Dịch vụ `kbm-core`: quản lý kỳ thi, bài tập SQL, quản lý bài nộp, chống gian lận, thống kê. Là dịch vụ ôm đồm (God Service) tạm thời chứa cả logic thi cử (mục tiêu tương lai sẽ tách rời thành `kbm-exam`).
`kbm-judge-sql`	SQL Practice	Điều phối chấm SQL: nhận thông điệp từ Kafka, phân bổ cho các sandbox tương ứng với từng hệ quản trị CSDL, tích hợp tính năng phản hồi tự động bằng trí tuệ nhân tạo (AI feedback).
`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, `kbm-sandbox-oracle`	SQL Practice	Bốn worker chạy mã SQL trong môi trường cô lập trên từng container CSDL — mỗi DBMS một worker riêng.
`kbm-judge-network`	Network Practice	Chấm điểm lập trình mạng Java (TCP/UDP/RMI/SOAP/REST/gRPC), hỗ trợ môi trường thực thi Java độc lập (standalone Java) và cơ chế tải lại động trình xử lý (dynamic handler reloading).

Tên service kỹ thuật	Bounded Context	Mô tả vai trò
`kbm-judge-code`	Code Practice	Chấm điểm thuật toán đa ngôn ngữ (C/C++/Java…). Thế hệ đầu tiên được phát triển nội bộ không có môi trường cô lập (sandbox); thế hệ thứ hai tích hợp DOMjudge để cung cấp môi trường cô lập mã nguồn mở.
`kbm-exam-client`	Examination & Proctoring	Ứng dụng máy tính để bàn (desktop) dành cho phòng máy: khóa môi trường thi (lockdown), danh sách IP cho phép, giám sát thi (proctoring) tại máy khách; sử dụng chung JWT của `kbm-auth`.
`kbm-academic`	Academic Management	Quản lý khóa học, bài tập dạng rubric, điểm danh, MCQ, syllabus/CLO, GitHub Classroom, đồ án tốt nghiệp.
`kbm-notification`	Communication	Đẩy thông báo thời gian thực (kết quả chấm bài) qua WebSocket; là consumer của Kafka.
`kbm-gateway`	Platform / Infra	Gateway của LMS Java (DevOps Lab có reverse proxy riêng, định tuyến theo hostname): định tuyến (path-based), giới hạn lưu lượng (rate limiting), xác minh JWT ở tầng biên (edge validation), cấu hình CORS.
`kbm-registry`	Platform / Infra	Dịch vụ đăng ký (Service registry) sử dụng Netflix Eureka để hỗ trợ Service Discovery giữa các ứng dụng Java.
`kbm-shared`	Platform / Infra (Shared Library)	Các thành phần dùng chung: thực thể cơ sở, đối tượng truyền dữ liệu (DTO), các tiện ích JWT, xử lý ngoại lệ tập trung (xem đánh giá trade-off của việc chia sẻ mã nguồn ở §2.5).
`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`	DevOps Practice	Nền tảng thực hành DevOps/Kubernetes được phát triển bằng Go với cấu trúc kho lưu trữ đơn (mono-repo multi-binary), bao gồm: API, máy chủ proxy đảo ngược (reverse proxy), bộ chấm điểm (grader), k3s và Sysbox.

Quy ước

Xuyên suốt các chương, để đảm bảo tính thực tiễn khi ánh xạ từ sơ đồ kiến trúc xuống mã nguồn hoặc cấu hình triển khai (Docker/Kubernetes), sách sử dụng trực tiếp các định danh kỹ thuật có tiền tố `kbm-*` này. Khi tự xây dựng các hệ thống Microservices, đội ngũ phát triển cũng nên duy trì một danh mục dịch vụ (service catalog) tương tự để giữ một ngôn ngữ chung (Ubiquitous Language) giữa các nhóm phát triển.

E.4.4. Tích hợp hệ thống bên ngoài

KBM tích hợp với nhiều hệ thống bên ngoài theo ba kiểu khác nhau, mỗi kiểu đặt ra thách thức kiến trúc riêng:

Bảng E.4: *Ba kiểu tích hợp hệ thống bên ngoài trong KBM*

Hệ thống	Kiểu tích hợp	Đặc điểm và thách thức
QLĐT của trường	Legacy — không có tài liệu API	Phải tự tìm hiểu API vì không có tài liệu. Cần Anti-Corruption Layer (ACL). Rủi ro cao: API có thể thay đổi không báo trước.
GitHub Classroom	API có tài liệu	REST API chuẩn, OAuth2 App. Tích hợp: đồng bộ bài tập, liên kết danh sách lớp (roster), theo dõi tiến độ.
Google / Microsoft OAuth2	Giao thức chuẩn	OpenID Connect chuẩn. Thách thức: gộp danh tính (identity merge) khi cùng một người dùng đăng nhập qua nhiều provider (xem Sự cố 4, §E.6).

E.4.5. Mô hình triển khai

Hai cụm triển khai đảm nhận các thành phần cốt lõi:

- **Docker Compose đơn máy (single-host):** Chạy các Java microservices và các thành phần dùng chung. Đây là đánh đổi tính sẵn sàng cao (high availability) lấy sự đơn giản trong vận hành.
- **k3s cluster:** Chạy DevOps Lab (các service Go) và các lab pod của sinh viên. Cluster k3s sử dụng ResourceQuota và LimitRange để giới hạn tài nguyên mỗi lab pod.

E.5. Trade-off và khoảng trống — bài học từ thực tế

Mỗi hệ thống phần mềm thực tế đều chứa trade-off — những quyết định *chấp nhận hạn chế ở khía cạnh này để đạt được lợi ích ở khía cạnh khác*. Với KBM, các trade-off không phải điều xấu hổ cần che giấu, mà là chất liệu tốt cho việc học kiến trúc: chúng ta học được nhiều hơn từ một quyết định "đủ tốt trong bối cảnh" so với một thiết kế lý tưởng trên lý thuyết.

Dưới đây là 25 trade-off của KBM tại thời điểm khảo sát (đầu năm 2026), được phân loại theo 3 mức nghiêm trọng. Mỗi mục kèm theo chương tham chiếu trong sách — nơi vấn đề được thảo luận chi tiết.

E.5.1. Nghiêm trọng (Critical) — 8 mục (tính đúng đắn / bảo mật)

Bảng E.5 liệt kê các trade-off ảnh hưởng tới tính đúng đắn và bảo mật.

Bảng E.5: *Trade-off mức nghiêm trọng — ảnh hưởng tính đúng đắn và bảo mật*

Trade-off	Mô tả ngắn	Chương
Thiếu idempotency phía consumer	API nộp bài đã có idempotency key (Sự cố 1), consumer Kafka chưa có — message lặp có thể bị chấm hai lần	Ch.5–6
Plain-header trust / thiếu workload identity	`X-User-*` không phải signed proof; service-to-service chưa nhận diện actor thống nhất	Ch.8–9
Secret management thô sơ	Một phần secret gắn cố định trong tệp cấu hình	Ch.9
Token lưu localStorage	Nếu có XSS, mã độc đọc được token	Ch.9
Judge code không sandbox (Gen 1)	Mã sinh viên chạy trực tiếp trên server	Ch.9–10
CORS allow all origins	Mọi trang web đọc được phản hồi API không kèm credentials (Ch.8)	Ch.8
Shared Database giữa service	Schema coupling → lỗi dây chuyền (cascading failure) tiềm ẩn	Ch.7
Bespoke JWT validation	Khác thư viện/quy tắc `alg`/issuer/audience → trôi lệch khi xác minh	Ch.8–9

E.5.2. Quan trọng (Major) — 11 mục (khả năng mở rộng / bảo trì)

Bảng E.6 liệt kê các trade-off ảnh hưởng tới khả năng mở rộng và bảo trì.

Bảng E.6: *Trade-off mức Quan trọng — ảnh hưởng khả năng mở rộng và bảo trì*

Trade-off	Mô tả ngắn	Chương
Shared library coupling	`kbm-shared` tạo coupling compile-time giữa các service	Ch.2–4
Gọi đồng bộ thiếu khả năng chịu lỗi (fault tolerance)	Không có circuit breaker, chính sách retry	Ch.4
Thiếu API versioning	API thay đổi có thể phá vỡ consumer	Ch.3
God Service	`kbm-core` quá nhiều trách nhiệm	Ch.2
Entity trùng lặp giữa service	Cùng khái niệm, khác model → trôi lệch dữ liệu	Ch.2–7
God Class ~1400 dòng	Class vi phạm Single Responsibility	Ch.7
Thiếu domain event	Không phát sự kiện khi trạng thái thay đổi → coupling ẩn	Ch.5

Trade-off	Mô tả ngắn	Chương
Thiếu database migration tool	JPA auto-DDL trong production → rủi ro schema drift	Ch.7–10
Kafka single-node	Không có replication → single point of failure	Ch.5
Correlation ID chưa xuyên suốt	Khó truy vết (trace) request qua nhiều service	Ch.11
Docker Compose single-host	Không có HA → downtime khi VPS gặp sự cố	Ch.12

E.5.3. Nhỏ (Minor) — 6 mục (cải thiện chất lượng)

Bảng E.7 liệt kê các điểm cải thiện chất lượng.

Bảng E.7: *Trade-off mức nhỏ — cải thiện chất lượng*

Trade-off	Mô tả ngắn	Chương
Thiếu caching layer	Mọi request đều truy vấn DB trực tiếp	Ch.7
Chưa tự động sao lưu (backup)	Rủi ro mất dữ liệu khi sự cố	Ch.12
Polyglot complexity	Java + Go → đội ngũ cần thành thạo 2 ngôn ngữ	Ch.1–12
Mono-repo rebuild	Thay đổi 1 binary → build lại cả 3	Ch.2
DB queue thay vì message broker	DevOps Lab dùng DB polling thay Kafka → không replay	Ch.5
Xây hay mua bộ chấm code	Quyết định chuyển từ tự phát triển sang mã nguồn mở	Ch.3–10

Tại sao công bố trade-off?

Việc liệt kê 25 trade-off — bao gồm cả những vấn đề nghiêm trọng như "không có sandbox" hay "quản lý secret thô sơ" — là có chủ đích. Mục tiêu giáo dục không phải trình bày một hệ thống lý tưởng (điều không tồn tại trong thực tế), mà cho thấy *mỗi quyết định đều có bối cảnh*. Trade-off Shared DB là ví dụ: với team 1–2 người, nhóm chưa cần tách máy chủ database nhưng vẫn cần ranh giới schema và quyền truy cập — shared DB là lựa chọn thực dụng, kèm theo kế hoạch tách dần khi đủ điều kiện (xem chi tiết trong Chương 7).

E.6. Sự cố thực tế và bài học

Bốn sự cố dưới đây đã xảy ra trong quá trình vận hành KBM. Mỗi sự cố được trình bày theo cấu trúc: **Vấn đề** → **Nguyên nhân gốc** → **Cách xử lý** (hoặc hướng xử lý) → **Bài học kiến trúc**.

E.6.1. Sự cố 1: Bài nộp trùng lặp trong kỳ thi

Vấn đề. Trong một kỳ thi SQL online, một số bài nộp bị trùng lặp — cùng một sinh viên, cùng một đề bài, cùng nội dung, nhưng xuất hiện hai lần trong hệ thống. Kết quả chấm bị ảnh hưởng.

Nguyên nhân gốc. Thiếu idempotency key trong API nhận bài nộp. Khi mạng timeout, client tự động retry → server nhận cùng một request hai lần và xử lý cả hai như request mới.

Cách xử lý. Thêm idempotency key (UUID) cho tất cả thao tác ghi quan trọng. Server kiểm tra key trước khi xử lý: nếu đã tồn tại → trả lại kết quả cũ, không tạo bản ghi mới. Phép kiểm tra phải nguyên tử: key được ghi với ràng buộc `UNIQUE` trong cùng transaction với bản ghi bài nộp, để hai lần retry đến cùng lúc không cùng lọt qua.

Bài học kiến trúc. Idempotency không phải thứ "có thì tốt" — đây là yêu cầu bắt buộc cho mọi thao tác ghi trong hệ thống phân tán. Mạng *sẽ* timeout, client *sẽ* retry. Xem chi tiết tại Chương 5 (Idempotent Consumer) và Chương 6 (Saga Pattern).

E.6.2. Sự cố 2: Dữ liệu không nhất quán giữa các bảng

Vấn đề. Hai service (`kbm-core` và `kbm-academic`) sử dụng chung database nhưng tạo các bảng trùng lặp cho cùng một khái niệm nghiệp vụ. Ví dụ: `class_room` và `as_course` cùng lưu thông tin khóa học. Dữ liệu giữa hai nhóm bảng dần trôi dạt (drift) — cập nhật ở bảng này không phản ánh ở bảng kia, dẫn đến sinh viên thấy thông tin khóa học khác nhau tùy theo tính năng đang sử dụng.

Nguyên nhân gốc. Hai giai đoạn phát triển (thực tế là cùng tác giả nhưng ở hai thời điểm khác nhau) xây dựng entity mapping độc lập, tạo ra 9 điểm xung đột: 3 cặp bảng trùng lặp, 3 bảng cùng tên nhưng mapping khác, 3 bảng bị ghi từ nhiều service.

Hướng xử lý. Chuyển đổi 3 giai đoạn: (1) Chuẩn hóa entity dùng chung qua `@Immutable` + `ReadOnlyRepository` trong `kbm-shared`; (2) Gộp bảng trùng lặp (`class_room` + `as_course` → `sys_course`); (3) Thiết lập quy ước quyền sở hữu (ownership): tiền tố `sys_` cho bảng dùng chung, `as_` cho bảng của nghiệp vụ bài tập, và mỗi khái niệm chỉ có một bên ghi (single writer).

Bài học kiến trúc. Shared database không tự động gây vấn đề — vấn đề nằm ở *thiếu quy ước quyền sở hữu* rõ ràng. Khi nhiều service cùng ghi vào cùng một khái niệm mà không có "nguồn sự thật duy nhất" (single source of truth), trôi lệch dữ liệu là hệ quả tất yếu. Xem chi tiết tại Chương 7 (Quản lý dữ liệu) và Chương 2 (Bounded Context).

E.6.3. Sự cố 3: Multi-instance Judge — định tuyến sai

Vấn đề. Khi mở rộng Network Judge (`kbm-judge-network`) lên nhiều instance, giao thức chấm 2 pha gặp lỗi: Bước 1 (nhận đề bài, sinh request ID) được xử lý bởi Instance A; Bước 2 (gửi bài giải kèm request ID) được chuyển đến Instance B. Instance B không biết request ID từ Instance A → từ chối bài nộp.

Nguyên nhân gốc. Giao thức chấm *stateful* (lưu request ID trong bộ nhớ của instance) nhưng hạ tầng lại mở rộng theo kiểu *stateless* (load balancer phân bổ ngẫu nhiên). Hai mô hình xung đột nhau.

Hướng xử lý. Có ba hướng: session affinity (sticky session) tại load balancer, shared state qua Redis hoặc database, hoặc thiết kế lại giao thức thành stateless. Sticky session chỉ che triệu chứng: state vẫn nằm trong bộ nhớ của instance, instance dừng là mất. Shared state gỡ được state khỏi instance nhưng thêm một phụ thuộc. Hướng thứ ba giải quyết tận gốc: bước gửi bài giải tự mang đủ ngữ cảnh chấm (self-contained) dưới dạng một token do judge ký (ví dụ bằng HMAC) và có hạn dùng, nhờ đó bất kỳ instance nào cũng xác minh và xử lý được yêu cầu mà không cần nhớ request ID từ bước trước, còn sinh viên không sửa được ngữ cảnh.

Bài học kiến trúc. Stateful protocol + stateless scaling = xung đột. Khi thiết kế giao thức giao tiếp giữa các service, cần *giả định rằng bất kỳ instance nào cũng có thể xử lý bất kỳ request nào* — đây là nguyên tắc stateless design. Nếu buộc phải giữ state, cần đưa state ra một kho dùng chung (shared store).

E.6.4. Sự cố 4: Gộp danh tính (Identity Merge) — khủng hoảng danh tính

Vấn đề. Khi mở rộng OAuth2 (thêm Microsoft bên cạnh Google), phát hiện nhiều người dùng có 2–3 tài khoản riêng biệt cho cùng một con người — đăng nhập qua Google tạo tài khoản A, qua Microsoft tạo tài khoản B, qua email/password tạo tài khoản C. Dữ liệu học tập (điểm, bài nộp, lịch sử) bị phân tán qua nhiều danh tính.

Nguyên nhân gốc. Thiếu cơ chế *liên kết danh tính* (identity linking) — hệ thống tạo tài khoản mới cho mỗi provider mà không kiểm tra email trùng. Đây là vấn đề phổ biến khi thêm OAuth2 provider vào hệ thống đã có sẵn người dùng.

Hướng xử lý. Liên kết các tài khoản cùng email đã xác thực thành một danh tính duy nhất và trỏ khóa ngoại của dữ liệu liên quan về danh tính chính, kèm bước đối soát thủ công cho các trường hợp biên (ví dụ cùng một người nhưng dùng các email khác nhau). Email chỉ là tín hiệu để gợi ý liên kết, vì định danh ổn định của nhà cung cấp là `iss` + `sub`, nên cần xác minh người dùng kiểm soát cả hai tài khoản. Mục tiêu là không mất dữ liệu học tập (điểm, bài nộp, lịch sử) khi gộp.

Bài học kiến trúc. Quản lý danh tính trong hệ thống nhiều nhà cung cấp cần được thiết kế từ đầu với khái niệm *identity* tách biệt khỏi *credential*. Một người dùng = một identity, có thể liên kết nhiều credential (email, Google, Microsoft). Xem chi tiết tại Chương 2 (BC discovery cho Identity & Access) và Chương 9 (Bảo mật).

E.7. Ánh xạ giữa case study và các chương

Mỗi chương trong sách đều tham chiếu đến ít nhất một khía cạnh của KBM. Bảng dưới đây tổng hợp ánh xạ giữa nghiệp vụ, chương, pattern/bài học, và sự cố liên quan (nếu có).

Bảng E.8: *Ánh xạ nghiệp vụ KBM ↔ chương sách ↔ sự cố thực tế*

Chương	Nghiệp vụ KBM liên quan	Pattern / bài học chính	Sự cố
Ch.1	Tổng quan hệ thống, từ các monolith rời rạc → microservices	Monolith Hell, Fast Flow, Quality Attribute	—
Ch.2	Khám phá 7+2 BC, God Service, thực thể trùng lặp	DDD, Bounded Context, Context Map	#2, #4
Ch.3	Multi-protocol judge, DOMjudge ACL, exam-client multi-client	API Design, “Xây hay mua” (build vs buy), Versioning	—
Ch.4	Gọi `kbm-judge-sql` qua OpenFeign, load balancing, tích hợp QLĐT, khả năng chịu lỗi	Sync Communication, Circuit Breaker, ACL	#3
Ch.5	Kafka cho chế độ thi, DB queue DevOps Lab, idempotency	Async Messaging, Idempotent Consumer	#1
Ch.6	Submit-Judge implicit saga, bài nộp trùng lặp	Saga Pattern, Compensating Transaction	#1
Ch.7	Shared DB, JPA auto-DDL vs golang-migrate, data drift	Database per Service, Shared DB trade-off	#2
Ch.8	Spring Cloud Gateway + Go hostname-based routing, CORS	API Gateway, Rate Limiting, làm sạch request ở biên và signed identity propagation	—
Ch.9	SSO đa nền tảng, OAuth2, rủi ro thiếu sandbox, khóa môi trường thi	Tiered Trust, JWT/JWKS, RBAC, workload identity và Secret Management	#4
Ch.10	Lộ trình chuyển đổi, Strangler Fig, xây → mua DOMjudge	Monolith → Microservices, Strangler Fig	—
Ch.11	Correlation ID, Prometheus, Promtail→Loki→Grafana	Observability, Distributed Tracing	—
Ch.12	Docker Compose vs k3s, CI/CD, RBAC, DevOps Lab infra	Triển khai, Container Orchestration, IaC	—